

Servicio: seed_data.py (NUEVO)

Archivo: `services/seed_data.py` · Documento técnico-pedagógico · Generado 2026-05-20 13:46

Carga datos de demostración al iniciar la aplicación: **60 citas precargadas** distribuidas entre 4 empresas y 13 horarios nocturnos. Esto permite tener una demo realista sin tener que registrar todo a mano.

Por qué se creó este archivo

La presentación del piloto dura máximo 20 minutos. No hay tiempo de registrar 60 citas manualmente. Este módulo genera datos sintéticos pero realistas: nombres peruanos, DNIs únicos, empresas con perfiles típicos y riesgos asociados a su rubro.

Las 4 empresas

Cada empresa tiene un perfil distinto. Esto se logra con un diccionario que define el tipo de personal, si es trabajo de mina y qué riesgos son típicos en ese rubro:

```
EMPRESAS = [  
    {  
        "nombre": "MINERA ANDINA SAC",  
        "es_mina": True,  
        "perfil_default": "Mina",  
        "riesgos_tipicos": [  
            "Trabajo en altura (> 1.80m)",  
            "Exposición a Radiaciones",  
            "Operadores de maquinaria pesada",  
        ],  
    },  
    {  
        "nombre": "CONSTRUCTORA PERU SA",  
        "es_mina": False,  
        "perfil_default": "Operativo",  
        "riesgos_tipicos": [  
            "Trabajo en altura (> 1.80m)",  
            "Espacios confinados",  
            ...  
        ],  
    },  
    # ENERGIA SUR SAC: químicos, radiaciones, espacios confinados  
    # AGRO EXPORT SA: administrativo, sin riesgos  
]
```

Cómo se generan las 60 citas

El generador recorre los nombres y empresas en orden circular para garantizar exactamente 15 citas por empresa. Para cada cita: elige tipo de examen al azar, asigna riesgos típicos del rubro de la empresa, construye el protocolo correspondiente, y busca la primera hora libre del día de hoy.

```
def cargar_datos_demo(total=60, seed=42):  
    if gestion_citas.lista_citas:  
        return 0 # no recargar si ya hay datos  
  
    random.seed(seed) # semilla fija = mismos datos cada vez  
    fecha_hoy = date.today().strftime("%d/%m/%Y")  
  
    creadas = 0  
    while creadas < total and intentos < total * 4:  
        empresa = EMPRESAS[creadas % len(EMPRESAS)] # rotamos empresas  
        nombre = " ".join(NOMBRES[creadas % len(NOMBRES)])  
        ...  
        proto = construir_protocolo(opcion, es_mina, riesgos)
```

```

# Buscar primera hora con cupo del día de hoy
for h in horas_orden:
    if gestion_citas.validar_tope_atenciones(fecha_hoy, h):
        hora_libre = h
        break
...
gestion_citas.lista_citas.append(cita)
creadas += 1

```

Para principiantes: `random.seed(42)` fija una **semilla** para el generador aleatorio. Eso significa que aunque se usa `random.choice()` y `random.randint()`, los resultados son los **mismos cada vez que ejecutas la app**. Es útil para demos: siempre obtienes la misma demo, pero los datos parecen 'aleatorios' al observador.

Integración en la app

`cargar_datos_demo()` se invoca una sola vez al iniciar la aplicación, desde `ui_simple.launch_simple_app()`. Si la lista de citas ya tiene contenido (porque se llamó antes), no hace nada para evitar duplicados.

```

def launch_simple_app():
    from services.seed_data import cargar_datos_demo
    cargar_datos_demo()          # <-- carga las 60 citas de demo
    root = tk.Tk()
    app = TBMedicSimpleApp(root)
    root.mainloop()

```